

# 2

Entwicklung und  
Umsetzung von Algorithmen

**Teil 2 der Abschlussprüfung**

## Allgemeine Korrekturhinweise

Die Lösungs- und Bewertungshinweise zu den einzelnen Handlungsschritten sind als Korrekturhilfen zu verstehen und erheben nicht in jedem Fall Anspruch auf Vollständigkeit und Ausschließlichkeit. Neben hier beispielhaft angeführten Lösungsmöglichkeiten sind auch andere sach- und fachgerechte Lösungsalternativen bzw. Darstellungsformen mit der vorgesehenen Punktzahl zu bewerten. Der Bewertungsspielraum des Korrektors (z. B. hinsichtlich der Berücksichtigung regionaler oder branchenspezifischer Gegebenheiten) bleibt unberührt.

Zu beachten ist die unterschiedliche Dimension der Aufgabenstellung (nennen – erklären – beschreiben – erläutern usw.).

Für die Bewertung gilt folgender Punkte-Noten-Schlüssel:

|        |   |                 |                |        |       |                |                |
|--------|---|-----------------|----------------|--------|-------|----------------|----------------|
| Note 1 | = | 100 – 92 Punkte | Note 2         | =      | unter | 92 – 81 Punkte |                |
| Note 3 | = | unter           | 81 – 67 Punkte | Note 4 | =     | unter          | 67 – 50 Punkte |
| Note 5 | = | unter           | 50 – 30 Punkte | Note 6 | =     | unter          | 30 – 0 Punkte  |

## 1. Aufgabe (30 Punkte)

a) 12 Punkte

```
List<KuhData> erzeugeKuhData(List<ProdData> prodData) {  
    List<KuhData> kuhData = new List<KuhData>()  
    foreach (ProdData pd in prodData) {  
        KuhData kd = new KuhData(pd.getName(), 0.0)  
        if(!kuhData.contains(kd)) {  
            kuhData.add(kd)  
        }  
    }  
    return kuhData  
}
```

Alternativ ohne contains

```
List<KuhData> erzeugeKuhData(List<ProdData> prodData) {  
    List<KuhData> kuhData = new List<KuhData>()  
    foreach (ProdData pd in prodData) {  
        boolean schonVorhanden = false  
        foreach (KuhData kd in kuhData) {  
            if (pd.getName() == kd.getName()) {  
                schonVorhanden = true  
            }  
        }  
        if (schonVorhanden == false) {  
            kuhData.add(new KuhData(pd.getName(), 0.0))  
        }  
    }  
    return kuhData  
}
```

b) 18 Punkte

```
List<KuhData> calcAvgLeistung(List<ProdData> prodData, Date start, Date
ende) {
    List<KuhData> kuhData = erzeugeKuhData(prodData)
    double sum
    int tage
    for (int i = 0, solange i < kuhData().size(), i++) {
        sum = 0.0
        tage = 0
        foreach (ProdData pd in prodData) {
            if (pd.getName() == kuhData.get(i).getName() &&
                pd.getDate().between(start, ende)) {
                sum = sum + pd.getLeistung()
                tage++
            }
        }
        kuhData.get(i).setAvgLeistung(sum / tage)
    }
    return kuhData
}
```

oder alternativ mit zwei foreach-Schleifen

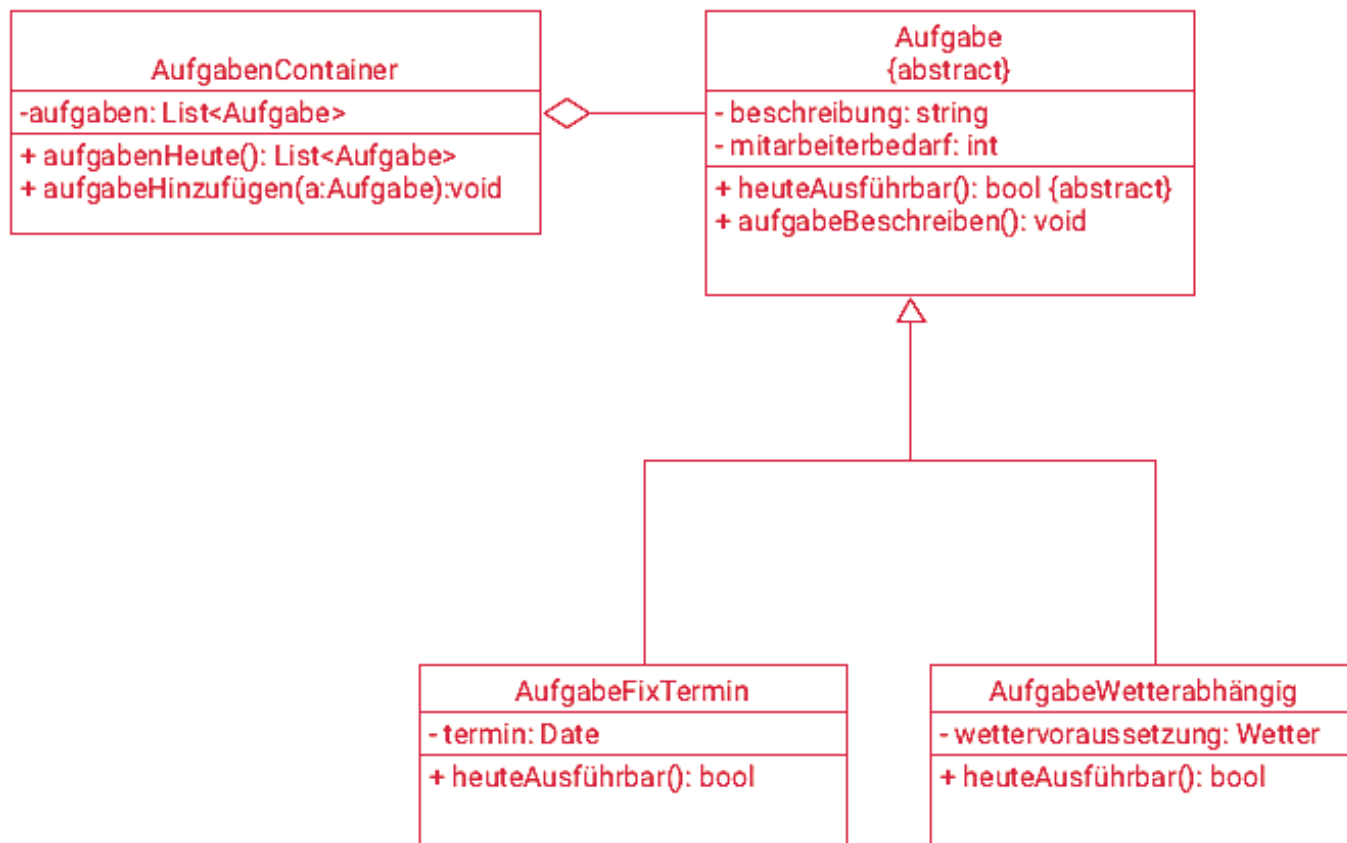
```
List<KuhData> calcAvgLeistung(List<ProdData> prodData, Date start, Date
ende) {
    List<KuhData> kuhData = erzeugeKuhData(prodData)
    double sum
    int tage
    foreach (KuhData kd in kuhData) {
        sum = 0.0
        tage = 0
        foreach (ProdData pd in prodData) {
            if (pd.getName() == kd.getName() &&
                pd.getDate().between(start, ende)) {
                sum = sum + pd.getLeistung()
                tage++
            }
        }
        kd.setAvgLeistung(sum / tage)
    }
    return kuhData
}
```

Andere Lösungen sind möglich

## 2. Aufgabe (20 Punkte)

a) 12 Punkte

Diagramm 12 Punkte (1 je Klasse, 1 für die Teil-Ganzes-Beziehung und 1 für die Vererbung, je 1 Punkt für die vollständigen Signaturen inkl. Sichtbarkeiten)



ba) 4 Punkte

(Hinweis: *override* ist für die volle Punktzahl nicht erforderlich, andere, ebenfalls korrekte Lösungen sind möglich)

```
public override List<Aufgabe> aufgabenHeute() {
    List<Aufgabe> liste = new List<Aufgabe>()
    foreach (Aufgabe aufgabe in Aufgaben) {
        if (aufgabe.heuteAusführbar()) {
            liste.add(aufgabe)
        }
    }
    return liste
}
```

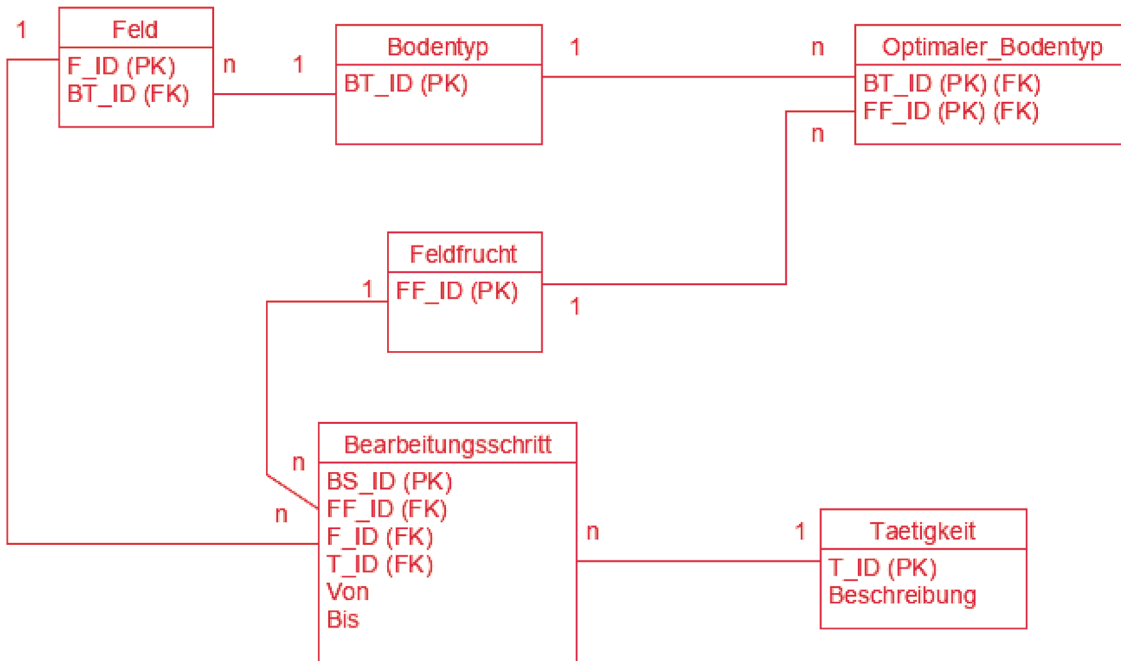
bb) 4 Punkte

Dieser Mechanismus ist *die späte Bindung* bzw. *late binding* (Bestandteil der Polymorphie).

Hierbei wird zur Laufzeit festgestellt, für welches konkrete Objekt (bzw. Objekttyp also Klasse) eine Methode aufgerufen wird und dann die zum Objekt (bzw. der Klasse) gehörende Methode aufgerufen. Zur Compilezeit kann diese Entscheidung nicht getroffen werden, da nicht bekannt ist, welche Objekte an welcher Position in der Liste stehen.

### 3. Aufgabe (25 Punkte)

a) 20 Punkte



Für jede Tabelle mit Primärschlüssel (1P) = 6 Punkte

Für die Attribute (von/bis) in der Bearbeitungsschritt-Tabelle (1P) = 1 Punkt

Für das Attribut Beschreibung in der Tabelle Tätigkeit (1P) = 1 Punkt

Für jeden Fremdschlüssel (1P) = 6 Punkte

Für jede Beziehung (1P) = 6 Punkte

b) 5 Punkte

Speicherbedarf pro Bild:

$50.000.000 \text{ Pixel/Bild} \cdot 24 \text{ Bit/Pixel} = 1.200.000.000 \text{ Bit} / 8 \text{ Bit/Byte} = 150.000.000 \text{ Byte}$

Speicherbedarf pro Woche:

$150.000.000 \text{ Byte/Bild} \cdot 500 \text{ Bilder} = 75.000.000.000 \text{ Byte}$

Speicherbedarf pro Jahr in TiB:

$75.000.000.000 \text{ Byte/Woche} \cdot 52 \text{ Wochen} = 3.900.000.000.000 \text{ Byte}$

$3.900.000.000.000 \text{ Byte} / 1024 / 1024 / 1024 / 1024 = 3,547 \text{ TiB} \approx 3,55 \text{ TiB}$

#### 4. Aufgabe (25 Punkte)

a) 3 Punkte

| Grundlegende Datenoperationen | Bedeutung          | DDL Data Definition Language | DML Data Manipulation Language | DQL Data Query Language |
|-------------------------------|--------------------|------------------------------|--------------------------------|-------------------------|
| <b>C (reate)</b>              | Anlegen, Erstellen | <b>CREATE</b>                | <b>INSERT</b>                  | -----                   |
| <b>R (ead)</b>                | Lesen              | -----                        | -----                          | <b>SELECT</b>           |
| <b>U (pdate)</b>              | Aktualisieren      | <b>ALTER</b>                 | <b>UPDATE</b>                  | -----                   |
| <b>D (elete)</b>              | Löschen            | <b>DROP</b>                  | <b>DELETE</b>                  | -----                   |

b) 8 Punkte

```
SELECT TK.TK_Kategorie,
       COUNT(TB.TB_ID) AS AnzahlTiere,
       MAX(TZI.TZI_Gewicht) AS SchwerstesTier,
       DateDiff(year, MAX(TB.TB_GebDat), NOW()) AS ÄltestesTier,
       AVG(DateDiff(year, TB.TB_GebDat, NOW())) AS DurchschnittAlter
FROM Tierkategorie AS TK
     LEFT JOIN Tierbestand AS TB ON TB.TB_TKID = TK.TK_ID
     LEFT JOIN TierZusatzInfo AS TZI ON TZI.TZI_TBID = TB.TB_ID
WHERE TB.TB_SchlachtDat IS NULL
```

ca) 10 Punkte

```
INSERT INTO Archiv_Tierbestand( A_TBID,
                                A_Tierkategorie,
                                A_ChipNr,
                                A_GebDat,
                                A_SchlachtDat,
                                A_MaxGewicht )
    SELECT
        TB_ID,
        TK_Kategorie,
        TB_ChipNr,
        TB_GebDat,
        TB_SchlachtDat,
        MAX(TZI.TZI_Gewicht)
    FROM Tierbestand AS TB
        LEFT JOIN Tierkategorie AS TK
            ON TK.TK_ID = TB.TB_TKID
        LEFT JOIN TierZusatzInfo AS TZI
            ON TZI.TZI_TBID = TB.TB_ID
    WHERE TB_SchlachtDat IS NOT NULL
    GROUP BY TB_ID,
        TK_Kategorie,
        TB_ChipNr,
        TB_GebDat,
        TB_SchlachtDat;
```

cb) 4 Punkte

```
DELETE FROM Tierbestand

    WHERE TB_ID IN (SELECT A_TBID FROM Archiv_Tierbestand);

DELETE FROM TierZusatzInfo

    WHERE TZI_TBID IN (SELECT A_TBID FROM Archiv_Tierbestand);
```

